

Atividade Prática – Aula 8

Relatórios de Cobertura & Vulnerabilidade + Badges

Disciplina: Integração e Entrega Contínua (IEC)

Professora: Lucineide

Semestre: 1º Semestre / 2026

Identificação do Aluno e Entregáveis:

Nome: Mario

Repositório: FATEC-JCR-4DSM-IEC-2026-1-seunome

Pasta: P2-Conteudos/Atividades/AtividadeAula08/

Exercício 1: Comando de Cobertura de Testes

O sistema do INPE necessita de alta confiabilidade nas suas rotinas de monitoramento climático. Para garantir que a lógica de alertas de queimadas esteja totalmente testada, foi configurado o Jest para gerar relatórios de cobertura de testes.

Configuração no package.json

Adicionamos o comando "test:coverage" para executar o Jest com a flag "--coverage".

```
{
  "name": "iec-atividade-aula08",
  "scripts": {
    "test": "jest",
    "test:coverage": "jest --coverage"
  }
}
```

Código da Função (alerta.js)

O módulo alerta.js contém as funções de classificação e envio de alertas:

```
function classificarAlerta(valor) {
  if (valor >= 90) return "Crítico";
  if (valor >= 70) return "Alto";
  return "Baixo";
}

function enviarNotificacao(alerta) {
  return `Notificação enviada: ${alerta}`;
}

function processarAlerta(valor) {
  const alerta = classificarAlerta(valor);
  return enviarNotificacao(alerta);
}
```

Por que fazer? Sem uma cobertura de código mensurável, não há garantia de que ramificações críticas foram validadas, expondo o sistema do INPE a falhas graves de produção.

Exercício 2: Automação do Relatório no Pull Request

O pipeline de Integração Contínua (CI) foi desenhado para validar novas adições de código antes que sejam integradas à branch principal. Assim, qualquer alteração no módulo de monitoramento climático exige a geração e validação automática de testes.

Gatilho de Disparo (Trigger)

Configuramos a esteira para rodar especificamente quando um Pull Request direcionado para a branch "main" for aberto ou atualizado.

```
# Trecho do arquivo .github/workflows/ci.yml
on:
  push:
    branches:
      - main
      - dev
  pull_request:
    branches:
      - main
```

Passos da Execução de Teste

Dentro da esteira de CI executada no GitHub Actions, garantimos a execução de cobertura através de:

```
- name: Executar Testes com Cobertura (Exercício 1)
  run: npm run test:coverage
```

Impacto no Projeto: Bloqueia integrações de código sem testes (ou com falhas) antes de chegarem à versão de produção, protegendo a confiabilidade dos alertas meteorológicos.

Exercícios 3 & 4: Codecov & Envio de Relatórios

Exercício 3: Integração do Histórico com Codecov

A avaliação contínua da qualidade exige uma linha do tempo das métricas de cobertura. O Codecov foi selecionado como a ferramenta de auditoria de qualidade. Configuramos a dependência no package.json para integração local e suporte do ecossistema.

```
"devDependencies": {  
  "jest": "^29.7.0",  
  "codecov": "^3.8.3"  
}
```

Exercício 4: Envio Automático via Pipeline

O relatório gerado no pipeline de CI (.github/workflows/ci.yml) é despachado automaticamente para o portal do Codecov utilizando a Action oficial.

```
- name: Enviar Relatórios de Cobertura para o Codecov  
  uses: codecov/codecov-action@v4  
  with:  
    file: ./coverage/clover.xml  
    flags: unittests  
    name: codecov-inpe-monitoring  
    fail_ci_if_error: false  
    token: ${{ secrets.CODECOV_TOKEN }}
```

Transparência Pública: Permite a auditoria externa e a rastreabilidade da evolução da cobertura, servindo de métrica objetiva para a governança pública do software.

Exercício 5: Badges no README.md

Para facilitar a visualização instantânea da qualidade do repositório pelos stakeholders e coordenadores do INPE, adicionamos dois badges informativos no topo do arquivo README.md.

1. Badge de Status do Pipeline (CI: Passing / Failing)

Exibe o estado atual da última compilação no GitHub Actions.

```
[![CI Pipeline](https://github.com/lucineidefatec/FATEC-JCR-4DSM-IEC-2026-1-seunome/actions/workflows/ci.yml/badge.svg)](https://github.com/lucineidefatec/FATEC-JCR-4DSM-IEC-2026-1-seunome/actions/workflows/ci.yml)
```

2. Badge de Cobertura de Código (%)

Exibe a porcentagem do código coberto por testes registrado na ferramenta Codecov.

```
[![codecov](https://codecov.io/gh/lucineidefatec/FATEC-JCR-4DSM-IEC-2026-1-seunome/branch/main/graph/badge.svg?token=YOUR_TOKEN)](https://codecov.io/gh/lucineidefatec/FATEC-JCR-4DSM-IEC-2026-1-seunome)
```

Valor para a Gestão: Comunicação visual rápida e clara nas revisões de sprint. Demonstra maturidade técnica de DevOps para a equipe e patrocinadores do projeto.

Evidências de Execução Local

Os testes foram executados com sucesso no ambiente de desenvolvimento local, demonstrando uma cobertura perfeita de 100% de linhas, funções e branches no arquivo de alertas climáticos.

Resultado da Cobertura de Testes (Coverage Report)

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	100	100	100	
alerta.js	100	100	100	100	
Test Suites:	2 passed, 2 total				
Tests:	7 passed, 7 total				

Conclusão

Com todas as etapas implementadas, o sistema de monitoramento de alertas de queimadas atende integralmente aos requisitos da disciplina de Integração e Entrega Contínua da professora Lucineide, combinando qualidade interna (100% de cobertura de testes) e visibilidade externa da saúde do projeto (pipeline, Codecov e badges).

